PayStruct: ISO 20022 Address Structuring Engine

Oct 7, 2026 · @Neeraj

Overview

PayStruct converts the free-text postal addresses inside ISO 20022 payment messages into the structured format the payments industry is moving to, validates the result, and explains every failure with a cited rule.

The problem. For decades, bank payment messages carried addresses as loose text lines. That is ending. Swift's Standards Release 2026 requires town and country in designated fields for all agents and parties in CBPR+ payment messages, with only structured or hybrid addresses accepted (Swift). That was due in November 2026. On 27 August 2026 Swift deferred it, because large parts of the industry in every region still could not comply. Swift will set the new timing by December 2026 at the latest (Swift news). CHAPS had a similar November 2026 deadline, so check the Bank of England's current position before quoting it. The requirement itself still stands, and banks and fintechs still hold millions of counterparty records as free text that must be split into street, building number, postcode, town and country.

What PayStruct does. It reads a payment message, finds every address, and structures it with deterministic rules first. An LLM handles only what the rules cannot. Every output is checked against the schema and the source text, low-confidence results go to a human review queue, and validation failures are explained with the exact rule that was broken.

Who it is for.

- Payment operations teams at banks and fintechs that send cross-border or CHAPS payments.
- As a portfolio project: hiring managers at banks and fintechs in Dublin, London and Toronto, and AI Engineer teams who care about reliable LLM systems.

Why it is not generic. It solves a real, unfinished industry problem in a domain few portfolio projects touch. Swift itself has released a free, open-source AI address-structuring model (Swift, GitHub). That proves the problem matters, and it means PayStruct must not be just another parser. PayStruct's difference is the full pipeline

around the parsing: message-level validation, a guarantee that no value is invented, confidence-based human review, cited explanations, streaming, and a public benchmark that includes Swift's own model as a baseline.

PayStruct is a working name. Check GitHub and trademarks before publishing under it.

Goals, non-goals and success metrics

The goal is a working, measured engine for two message types and a defined subset of address rules, not a bank-certified product.

Goals

1. Structure addresses in pacs.008 and pain.001 messages into ISO 20022 structured address elements.
2. Validate every output against the XSD schema and a documented subset of CBPR+ address rules.
3. Resolve as much as possible with deterministic rules, and use the LLM only for the remainder.
4. Verify every LLM output: no value may appear in the output that is not in the input.
5. Route uncertain results to a human review queue instead of guessing.
6. Explain validation failures with a retrieved, cited rule.
7. Process batches through Kafka and report throughput.
8. Publish an evaluation harness and its results, including where the system fails, with Swift's open-source structuring model as a baseline.

Non-goals

- No real customer or bank data. Everything is synthetic.
- No claim of CBPR+ certification. PayStruct implements a stated subset of address rules.
- No Swift network connectivity, sanctions screening or payment execution.
- No message types beyond pacs.008 and pain.001 in version 1.
- No polished product UI beyond a review screen and a demo page.

Success metrics

The targets below are starting points. Revise them after the week 2 rules-only baseline, and only ever report numbers that come from an evaluation run.

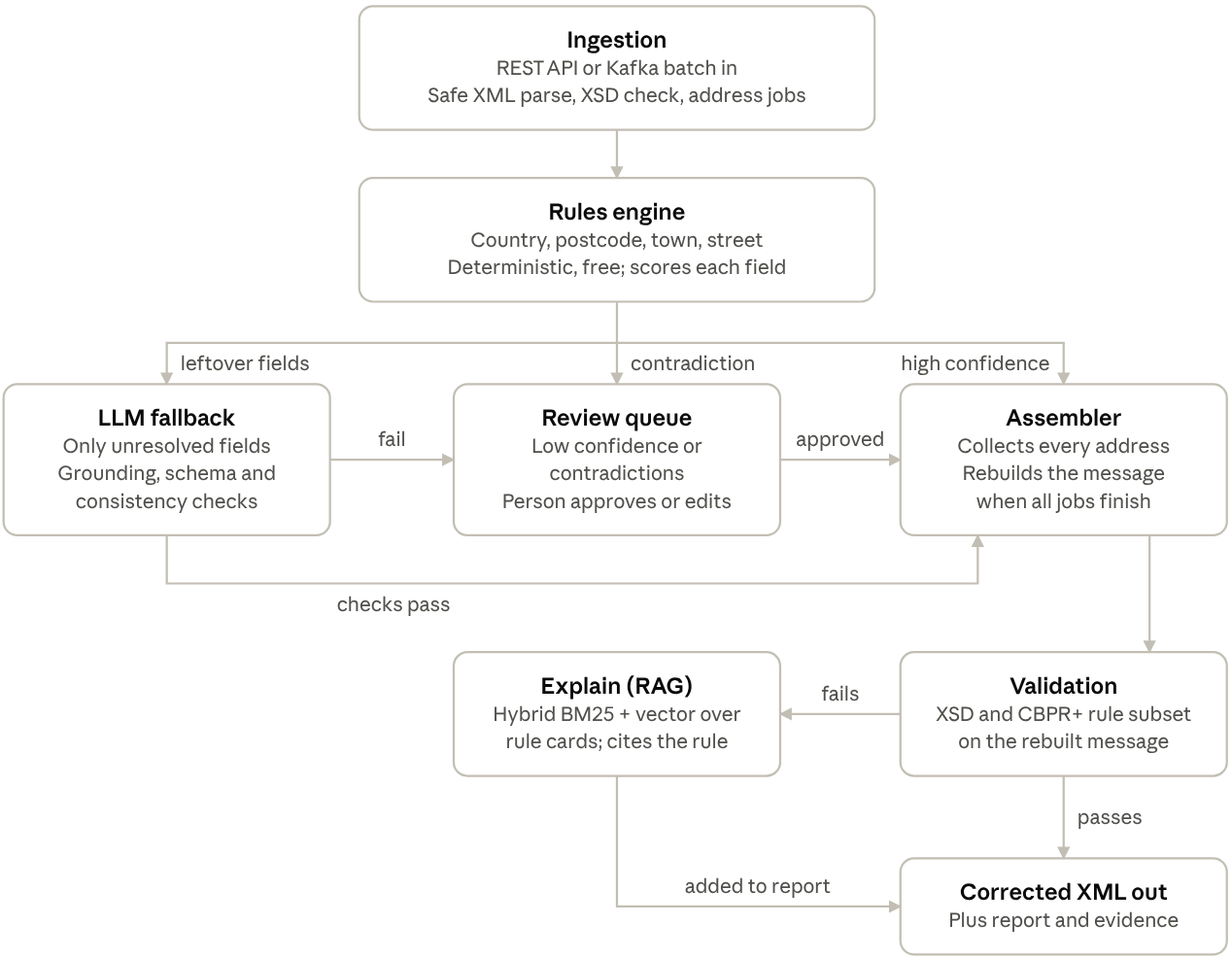
Metric	Initial target	Measured on
Hallucinated field values	0 (enforced by design)	Full test set
Country accuracy	99% or higher	Full test set
Town accuracy	95% or higher	Full test set
Address exact match (all fields correct)	90% or higher	Full test set
Error rate among auto-accepted addresses	1% or lower	Full test set
Share routed to human review	Report it, minimise it	Full test set
Share resolved by rules alone	Report it	Full test set
p95 latency per address, rules path and LLM path	Report both	Benchmark run
LLM cost per 1,000 addresses	Report it	Benchmark run
Kafka pipeline throughput (messages per second)	Report with hardware stated	Local benchmark
Retrieval recall at 3 for failure explanations	90% or higher	Failure set

System architecture

Every address takes one of three routes after the rules engine: straight to the assembler, through the LLM with checks, or to a person. Nothing reaches the output without passing validation.

Rules go first, the LLM fills gaps, and every result is checked

In batch mode each arrow is a Kafka topic; the API runs the same steps synchronously



PayStruct architecture · 8 components

Read it top to bottom. An address only meets the LLM when the rules leave fields unresolved, and an LLM answer that fails its checks goes to a person, never to the output. Postgres holds jobs, the review queue, rule cards and evaluation runs, and Redis caches repeat addresses; both sit behind every stage, so they are left out of the picture.

Domain primer: ISO 20022 addresses

An ISO 20022 payment is an XML message, and every party in it can carry a postal address block called `PstlAdr`. PayStruct's whole job is to fill that block correctly.

The two message types in scope

Message	What it is	Who sends it	Parties with addresses
pacs.008	FI-to-FI customer credit transfer	One bank to another	Debtor, creditor, ultimate debtor, ultimate creditor, and agents without a BIC
pain.001	Customer credit transfer initiation	A company to its bank	Debtor, creditor, ultimate parties, creditor agent

For agents (banks), a BIC alone remains a valid option instead of a name and address (Swift). So most of the hard work is on debtor and creditor addresses.

Address elements PayStruct fills

Element	Meaning	Max length
`Dept`, `SubDept`	Department, sub-department	70
`StrtNm`	Street name	70
`BldgNb`	Building number	16
`BldgNm`	Building name	35
`Flr`, `Room`	Floor, room	70
`PstBx`	PO box	16
`PstCd`	Postcode	16
`TwnNm`	Town	35
`TwnLctnNm`, `DstrctNm`	Town location, district	35
`CtrySubDvsn`	State, province or county	35
`Ctry`	Country, ISO 3166 two-letter code	2
`AdrLine`	Free-text line	70 each

These lengths come from the PostalAddress24 type used by current CBPR+ message versions. Confirm them against the XSD you download before coding the limits.

Three address forms

- **Unstructured:** only `AdrLine` free text. This is what is being retired.

- **Structured:** only discrete elements, no `AdrLine`. This is the target.
- **Hybrid:** `TwnNm` and `Ctry` as structured fields, plus a small number of `AdrLine` entries for the rest. This is the fallback when an address cannot be fully structured. Check the exact hybrid limits in the current CBPR+ usage guidelines before coding them.

Before and after (synthetic example)

```
<!-- Before: unstructured -->
<Cdtr>
  <Nm>Harbour Lights Ltd</Nm>
  <PstlAdr>
    <AdrLine>Unit 4 Block B Docklands Business Park</AdrLine>
    <AdrLine>Dublin 1 D01 K2X7 Ireland</AdrLine>
  </PstlAdr>
</Cdtr>

<!-- After: structured -->
<Cdtr>
  <Nm>Harbour Lights Ltd</Nm>
  <PstlAdr>
    <BldgNm>Block B Docklands Business Park</BldgNm>
    <Room>Unit 4</Room>
    <PstCd>D01 K2X7</PstCd>
    <TwnNm>Dublin 1</TwnNm>
    <Ctry>IE</Ctry>
  </PstlAdr>
</Cdtr>
```

Even this small example needs judgment: is "Dublin 1" a town or a district, and where does "Unit 4" go? Those ambiguous cases are exactly what the rules, the LLM fallback and the review queue are for.

Three layers of validation

1. **XSD schema:** element names, order, lengths and patterns. Free and mechanical.
2. **Usage-guideline rules:** cross-element rules, such as town and country being mandatory. Implement a documented subset.
3. **PayStruct quality rules:** the country code exists, the postcode matches the country's format, and the town is plausible for that postcode.

Where the reference material comes from. ISO 20022 message schemas and definitions are published at iso20022.org. Swift's CBPR+ usage guidelines live in Swift MyStandards, which needs a free swift.com account. Do not copy restricted documents into your public repo. Write your own short rule summaries instead, each with a reference to the source.

Synthetic data and the labelled dataset

Every number PayStruct reports depends on this dataset, so it is built first and built carefully. All data is synthetic: no real customer, bank or employer data, ever.

How the generator works. It runs in reverse. It first creates the correct structured address (the ground truth), then renders it into messy free text the way real systems do. Because the answer is known before the mess is made, every example is labelled automatically.

1. **Create the truth.** Use Python and Faker with country locales to produce structured fields: street, building number, building name, floor, postcode, town, region and country.
2. **Render it badly.** Apply corruption functions that copy real-world mess:
   - Reordered parts, missing commas, everything on one line or spread over many.
   - Abbreviations: St, Rd, Ave, Str., Apt, Flat.
   - All caps, extra spaces, country name instead of code, or no country at all.
   - Postcode on the wrong line, split, or missing its space.
   - Typos at a controlled rate.
   - Lines over 70 characters, which must be split.
3. **Wrap it in a message.** A message builder places the address into a valid pacs.008 or pain.001 skeleton. Some messages also get message-level faults, like a missing country or too many lines.

Country coverage. Pick countries whose address formats differ in useful ways.

Country	Why it is included
Ireland (IE)	Eircodes, many addresses without street numbers, Dublin postal districts
United Kingdom (GB)	Complex postcodes, flats and building names
Germany (DE)	Building number after the street name

Country	Why it is included
France (FR)	Number before street, five-digit postcode before town
Netherlands (NL)	Postcodes like 1234 AB
Canada (CA)	Provinces, A1A 1A1 postcodes, English and French
United States (US)	States, ZIP and ZIP+4
India (IN)	Long addresses, landmarks, six-digit PIN codes

Dataset splits

Split	Size	Used for
Development	600	Building and tuning rules and prompts
Calibration	400	Setting confidence thresholds, choosing few-shot examples
Test (frozen)	1,000	Final reported numbers only. Never look at it while tuning.
Hand-written hard set	100	Addresses you write yourself, outside the generator's patterns

The hand-written set matters most for credibility. A rules engine tuned on generator output can learn the generator's habits and look better than it is. Real-looking cases you write by hand catch that, so always report its results separately.

Record format (JSONL, one address per line)

```
{"id": "ie-00042", "country": "IE", "difficulty": "hard",
 "tags": ["no_street_number", "postal_district"],
 "raw_lines": ["Unit 4 Block B Docklands Business Park", "Dublin 1 D01 K2X7
Ireland"],
 "gold": {"Room": "Unit 4", "BldgNm": "Block B Docklands Business Park",
          "PstCd": "D01 K2X7", "TwnNm": "Dublin 1", "Ctry": "IE"}}
```

Rules for the generator

- Fixed random seed, recorded in the repo, so anyone can regenerate the exact dataset.

- Difficulty tiers (easy, medium, hard) so results can be broken down by difficulty.
- A labelling guide that settles ambiguous cases once, such as whether a Dublin district is a town or a district. Without it, the labels contradict each other.

Ingestion, XML parsing and schema validation

The ingestion service turns an incoming XML message into a list of address jobs, and refuses anything unsafe or malformed before any processing starts.

Step by step

1. **Receive.** A message arrives through the REST API (one message) or through Kafka (batches). Each gets an ID and a SHA-256 hash of its content.
2. **Deduplicate.** If the hash was already processed, return the stored result. This is the same idempotency pattern as SmartMail.
3. **Parse safely.** Disable external entities and DTDs in the XML parser. This blocks XXE attacks, which banks test for, so mention it in interviews.
4. **Detect the message type** from the XML namespace, for example `urn:iso:std:iso:20022:tech:xsd:pacs.008.001.08`. Unknown types are rejected with a clear error.
5. **Validate the input** against the XSD. Record schema errors, but still try to process the addresses. Bad input is what PayStruct exists to fix.
6. **Extract address jobs.** For each party with a `PstlAdr` block, create one job.

The address job model

```
public record AddressJob(
    String messageId,
    String messageType,      // pacs.008 or pain.001
    PartyRole role,          // DEBTOR, CREDITOR, ULTIMATE_DEBTOR, ...
    String xmlPath,          // where the address sits in the message
    List<String> rawLines,   // the AdrLine values as received
    StructuredAddress existing, // any fields already structured
    String countryHint       // from the party's BIC or the agent's country,
if present
) {}
```

Implementation choices

Need	Choice	Why
Java classes from the schema	JAXB with `xjc` code generation	Typed access, generated from the official XSD
Very large batch files	StAX streaming parser	Reads element by element without loading the whole file into memory
Schema validation	`javax.xml.validation`	Built into Java, no extra dependency
Fast path	Skip addresses already fully structured	Validate only, never rewrite what is already correct

A useful bank-specific signal. Characters 5 and 6 of a BIC are the country code. If a party's bank has a BIC, that country is a strong hint for the address. It is only a hint, because a customer can bank in a different country from where they live.

Deterministic rules engine

The rules engine is a chain of small, testable steps that resolves the easy and medium addresses for free, and records exactly which fields it is sure of. This is the QuerySense idea again: cheap deterministic work first, the LLM only for what is left.

The pipeline

1. **Normalise.** Unicode NFC, trim, collapse spaces. Keep the original text for output so accents and capitalisation survive. Work on a normalised copy for matching.
2. **Detect the country.** In order of trust: an explicit ISO code, a country name from a dictionary in several languages, a postcode pattern unique to one country, then the BIC hint.
3. **Extract the postcode** with a regex per country, for example UK `SW1A 1AA`, Irish Eircode `D01 K2X7`, Canada `K1A 0B1`, Germany five digits, Netherlands `1234 AB`, India six digits.
4. **Find the town** with a gazetteer lookup, using GeoNames city data (free, CC BY licence, credit it in the README). Search near the postcode first, usually the same or the next line.
5. **Split street and building number** using a per-country profile, because the order differs: `12 Main Street` in IE, GB, US and CA, but `Hauptstrasse 12` in DE and NL.

6. **Recognise the rest** with keyword lists: building names, floors, units, flats, rooms and PO boxes.
7. **Enforce limits.** Check every field against its maximum length. If the address cannot be fully structured, build a hybrid address instead of failing.
8. **Score confidence** for every field, and decide the route.

Country profiles live in config, not code. Each country gets a YAML file, so adding a country means adding a file, not changing Java.

```
country: DE
postcode_regex: "^[0-9]{5}$"
building_number_position: after_street
street_suffixes: [strasse, str., weg, allee, platz, gasse]
region_field: CtrySubDvsn
```

Code structure. Every step implements one interface, so each can be unit tested alone and reordered without rewriting the others.

```
public interface AddressRule {
    String id();                        // e.g. "postcode.extract"
    ParseState apply(ParseState state); // returns the state with fields and
evidence added
}
```

`ParseState` holds the remaining unparsed tokens, the fields found so far, and the evidence for each field: which rule set it and why. That evidence powers the review screen and the explanations.

Routing after the rules run

Outcome	Example	What happens
All required fields found, high confidence	Clean UK address with postcode and town	Accepted, no LLM call
Some fields found, some tokens left over	Street unclear, town and country certain	LLM fills only the missing fields
Country or town not found	No postcode, unknown place name	Full LLM fallback

Outcome	Example	What happens
Rules contradict each other	Postcode says one country, text says another	Human review, no guessing

A baseline to beat. libpostal is a well-known open-source address parser. Run it as a separate baseline, alongside Swift's model, so your results compare against real tools rather than against nothing.

LLM fallback, confidence scoring and human review

The LLM only sees what the rules could not resolve, and nothing it returns is trusted until it passes checks that cannot be argued with. This section is the core of the project for AI Engineer interviews.

What the LLM receives

- The original address lines, and nothing else from the message. No names, account numbers or amounts are sent. Addresses are personal data under GDPR, so send the minimum. Say this in interviews.
- The fields the rules already found, marked as fixed.
- The country hint and that country's format notes.
- A required JSON output schema, enforced through tool use or structured output.

Model choice. Start with a small, fast model, such as Claude Haiku. Escalate to a larger model only when the small one fails the checks below. Measure both, because the cost and accuracy trade-off is a result worth reporting.

The checks every LLM answer must pass

1. **Grounding check (the key one).** Every value must be built from tokens that appear in the original text. The only exceptions are the country code and abbreviation expansions from a fixed allowlist, such as `St` to `Street`. Any invented value means the answer is rejected. This is how PayStruct guarantees zero hallucinated fields.
2. **Schema check.** Field lengths, a valid country code, and the postcode matching the country's regex.
3. **Consistency check.** Postcode, town and country must agree according to the gazetteer.
4. **Agreement check.** Where the rules found a field, the LLM must not contradict it.

A failed check is never silently fixed. It either triggers one retry with the error explained, or routes the address to review.

Confidence scoring. Each address gets a score from 0 to 1, combined from simple signals.

Signal	Raises confidence when
Source of each field	Set by a rule, not the LLM
Gazetteer match	Town and postcode agree
Leftover tokens	Nothing from the input is left unused
Self-consistency	Two LLM samples give the same answer
BIC country	Matches the parsed country

Set the thresholds on the calibration split, never on the test set. The aim is a measured promise like: of everything auto-accepted, at most 1% is wrong. Then report how many addresses had to go to review to keep that promise. That trade-off is exactly how a bank would judge the system.

The human review queue. A Postgres table of addresses waiting for a person, with status `PENDING`, `APPROVED` or `CORRECTED`. A reviewer sees the original text, the proposed fields and the evidence for each, then approves or edits. Every correction is saved as a new labelled example, which can later improve few-shot prompts. Keep those corrections out of the frozen test set.

Cost and reliability controls

- **Cache:** results keyed by a hash of the normalised address in Redis. The same counterparty appears again and again in real payment flows, so caching matters. This is the QuerySense pattern again.
- **Budget cap:** a per-batch spending limit, after which addresses go to review instead of the LLM.
- **Timeouts, retries with backoff, and a circuit breaker.** If the LLM provider is down, addresses go to review rather than failing the batch.

RAG explanation layer

RAG in PayStruct does one job: when a message fails validation, it finds the exact rule that was broken and explains it with a citation. It never parses addresses. That narrow job is what makes it measurable, and what makes it a real RAG project rather than a chatbot.

The corpus: rule cards you write

Each rule is one short card, written in your own words, with a reference to its source. One rule per card is also one chunk, so retrieval returns whole rules, not fragments.

```
rule_id: PS-ADDR-007
title: Town and country required for every party
applies_to: [pacs.008, pain.001]
element_path: PstlAdr/TwnNm, PstlAdr/Ctry
source: CBPR+ usage guidelines, structured address requirement
effective: deferred, new date to be set by Swift
summary: Every party address must carry the town and the two-letter country
code in their own fields, even in a hybrid address.
fix: Move the town and country out of AdrLine into TwnNm and Ctry.
```

Cover three kinds of card: schema rules (lengths, required elements), usage-guideline rules (the documented subset), and country format notes. Aim for 60 to 100 cards.

Retrieval: hybrid search

1. **Build the query from the failure itself:** the error code, the element path and the message type. A failure is structured data, so use it rather than vague text.
2. **Keyword search (BM25)** using Postgres full-text search. It finds exact element names like `TwnNm`, which embeddings handle badly.
3. **Vector search** using pgvector over card embeddings. It finds rules described in different words.
4. **Merge with reciprocal rank fusion**, filtered by message type and effective date.
5. **Return the top 3 cards.**

This is the hybrid BM25 plus vector design you planned for Incident Copilot, applied where it has a clear, testable purpose.

Generation with a citation guard

The LLM returns JSON with `rule_id`, `why_it_failed` and `suggested_fix`. The `rule_id` must be one of the retrieved cards, or the answer is rejected. So it can never cite a rule that was not retrieved.

How it is evaluated. Build a set of 100 failing messages, each labelled with the rule that should explain it.

Metric	What it tells you
Recall at 3	Was the right rule among the 3 retrieved cards?
Citation accuracy	Did the explanation cite the right rule?
Keyword-only versus hybrid	Did adding vector search actually help?
Faithfulness (spot check of 30)	Does the explanation say only what the card says?

The keyword-only versus hybrid comparison is a strong interview point, because it shows you measured whether the extra complexity was worth it.

Kafka pipeline, persistence and data model

Batches flow through Kafka topics, one topic per stage, so each stage scales and fails on its own. The slow, rate-limited LLM stage cannot hold up the fast rules stage.

Topics

Topic	Written by	Read by	Contains
`payments.raw`	API or batch upload	Ingestion	Raw XML messages
`addresses.jobs`	Ingestion	Rules engine	One address job per party
`addresses.llm`	Rules engine	LLM worker	Jobs the rules could not finish
`addresses.done`	Rules engine, LLM worker	Assembler	Structured results with confidence
`addresses.review`	Rules engine, LLM worker	Review service	Low-confidence or contradictory jobs
`payments.validated`	Assembler	Output and reporting	Rebuilt, validated XML
`*.retry` and `*.dlq`	Any stage	Retry worker, a person	Failed jobs with retry count and error

Design decisions to explain in interviews

- **Partition key is the message ID,** so all addresses from one message land on the same partition and the assembler can rebuild the message in order.
- **At-least-once delivery with idempotent writes.** Exactly-once is hard and not needed: every write is keyed by job ID, so processing a job twice gives the same result.
- **Retries with backoff, then a dead-letter topic.** After a fixed number of failures a job goes to the DLQ for a person to inspect, rather than blocking the partition.
- **Backpressure on the LLM.** The LLM worker group's concurrency is capped to fit the provider's rate limit. Jobs queue in Kafka instead of failing.
- **The assembler waits for all addresses of a message.** It tracks how many jobs each message created and rebuilds the XML only when all have finished or gone to review.

Running it locally. Docker Compose starts Kafka in KRaft mode, Postgres with pgvector, Redis, Prometheus and Grafana. Throughput benchmarks run here, and every reported number states the hardware it ran on.

Data model (Postgres)

```
CREATE TABLE messages (
  id            UUID PRIMARY KEY,
  content_hash  CHAR(64) UNIQUE NOT NULL,   -- idempotency
  message_type  TEXT NOT NULL,
  status        TEXT NOT NULL,               -- RECEIVED, PROCESSING, DONE,
NEEDS_REVIEW
  input_valid   BOOLEAN,
  received_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE addresses (
  id            UUID PRIMARY KEY,
  message_id    UUID REFERENCES messages(id),
  party_role    TEXT NOT NULL,
  raw_lines     JSONB NOT NULL,
  result        JSONB,                       -- structured fields
  evidence      JSONB,                       -- which rule or model set each
field
  method        TEXT,                        -- RULES, LLM, REVIEW
  confidence    NUMERIC(4,3),
  latency_ms    INTEGER,
  llm_cost_usd  NUMERIC(10,6)
);

CREATE TABLE review_items (
  address_id    UUID PRIMARY KEY REFERENCES addresses(id),
  status        TEXT NOT NULL,               -- PENDING, APPROVED, CORRECTED
  corrected     JSONB,
  reviewed_at   TIMESTAMPTZ
);

CREATE TABLE rule_cards (
  rule_id       TEXT PRIMARY KEY,
  body          TEXT NOT NULL,
  metadata      JSONB NOT NULL,
  search_text   TSVECTOR,                    -- BM25-style keyword search
  embedding     VECTOR(1024)                 -- set the size to your
embedding model
);

CREATE TABLE eval_runs (
  id            UUID PRIMARY KEY,
  git_sha       TEXT NOT NULL,
```

```
  config        JSONB NOT NULL,              -- model, prompt version,
thresholds
  metrics       JSONB NOT NULL,
  ran_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

The `eval_runs` table keeps every evaluation result with the exact code version and settings that produced it. That history lets you show progress over time and prove every number on your resume.

REST API and review screen

A small, documented REST API serves the demo and the review screen. The batch pipeline runs through Kafka behind it.

Endpoints

Method	Path	Does
POST	`/v1/addresses/structure`	Structures raw address lines with an optional country hint. The simplest demo call.
POST	`/v1/messages`	Takes one XML message and returns the corrected XML plus a report
GET	`/v1/messages/{id}`	Returns the status and result of a message
POST	`/v1/batches`	Uploads a file of messages and returns a batch ID
GET	`/v1/batches/{id}/report`	Returns batch counts, accuracy if labelled, latency and cost
GET	`/v1/review?status=PENDING`	Lists addresses waiting for review
POST	`/v1/review/{addressId}`	Approves or corrects a reviewed address
GET	`/v1/explain/{errorId}`	Returns the cited explanation for a validation failure

Document the API with OpenAPI through springdoc, and protect the public demo with an API key and rate limiting.

Example response from `/v1/addresses/structure`

```
{
  "form": "STRUCTURED",
  "fields": {"Room": "Unit 4", "BldgNm": "Block B Docklands Business Park",
             "PstCd": "D01 K2X7", "TwnNm": "Dublin 1", "Ctry": "IE"},
  "evidence": {"PstCd": "rule postcode.extract (Eircode pattern)",
               "BldgNm": "llm, grounded in line 1"},
  "confidence": 0.91,
  "route": "AUTO_ACCEPTED",
  "latency_ms": 412
}
```

The values above show the response shape only. They are not results.

The review screen. This is where your visualizer background pays off: make the system's reasoning visible, as JavaMem and SQLMem do.

- The original address lines on the left, with each token coloured by the field it was assigned to.
- The proposed fields on the right, each showing its confidence and whether a rule or the LLM set it.
- Leftover, unassigned tokens highlighted, because they are usually where errors hide.
- Approve, or edit a field and save the correction.
- A before and after view of the XML for the whole message.

Build it as a single HTML page, or in Vue since you have used it in production. Keep it simple: it exists to show judgment, not design skill.

The public demo page. A paste-an-address box that animates the address through each stage: rules, LLM if needed, checks, and the final route. It turns the architecture into something a recruiter understands in 20 seconds.

Evaluation harness and benchmarks

The evaluation harness is what separates PayStruct from a demo: it compares five systems on the same frozen test set and publishes the results, including the failures.

Systems compared

System	What it is	Why it is included
A. Rules only	The rules engine with no LLM	Shows how far free, deterministic work goes
B. libpostal	Open-source address parser	A widely known baseline
C. Swift's model	Swift's open-source AI structuring model	The industry's own baseline, the comparison that matters most
D. LLM only	The LLM on every address, with the same checks	Shows what rules first saves in cost and errors
E. PayStruct	Rules first, LLM fallback, checks, review routing	Your system

If Swift's model beats PayStruct on raw parsing accuracy, say so plainly. PayStruct can still win on what surrounds the parsing: zero invented values, calibrated review routing, message-level validation and explanations. An honest loss on one metric is more convincing than a suspicious win on all of them.

Metrics

- **Field level:** precision, recall and F1 for each element, such as `StrtNm`, `PstCd` and `TwnNm`.
- **Address level:** exact match, meaning every field correct.
- **Safety:** hallucinated values, the schema-valid rate, and the error rate among auto-accepted addresses.
- **Operations:** the share resolved by rules, the share sent to review, p50 and p95 latency per route, LLM cost per 1,000 addresses, and Kafka throughput.
- **Breakdowns:** every metric by country, by difficulty tier, and on the hand-written hard set separately.

The coverage versus accuracy curve. Plot what share of addresses is auto-accepted against the error rate among them as the confidence threshold moves. Pick the operating point from the calibration split. This one chart answers a bank's real question: how much can we automate safely?

Reproducibility rules

- Fixed seeds, a frozen test set, and temperature 0 for every LLM call.

- Every run is stored with its git commit, model name, prompt version and thresholds in `eval_runs`.
- The test set is never used for tuning. If you change something after seeing test results, say so in the write-up.
- The README results table is generated from `eval_runs`, never typed by hand.

Results table template. Fill it only from real runs.

System	Exact match	Hallucinated values	Error rate when auto-accepted	Sent to review	p95 latency	Cost per 1,000
A. Rules only						
B. libpostal						
C. Swift's model						
D. LLM only						
E. PayStruct						

Regression gate in CI. Every pull request runs the rules-only path on a 100-address sample, which costs nothing. If exact match drops by more than 2 points, the build fails. The full evaluation with LLM calls runs on demand, under a budget you set.

Tech stack and repository structure

The service is Java and Spring Boot because banks run on Java and it is your strongest stack; Python handles only the data generation and evaluation, where it is the natural tool.

Layer	Choice	Why
Service	Java 21, Spring Boot 3	Your production stack, and the bank standard
Messaging	Apache Kafka (KRaft), Spring Kafka	Staged pipeline, closes the Kafka gap on your resume
XML	JAXB with `xjc`, StAX, `javax.xml.validation`	Typed parsing, streaming, schema validation
Database	PostgreSQL 16 with pgvector	Data, review queue, keyword and vector search in one place

Layer	Choice	Why
Cache	Redis	Repeat-address cache, as in QuerySense
LLM	Anthropic API (Claude)	Tool use for enforced JSON output
Gazetteer	GeoNames city data	Free town and postcode reference
Baselines	libpostal, Swift's open-source model	Honest comparisons
Data and evaluation	Python 3.12, Faker, pandas	Generator, metrics, charts
Testing	JUnit 5, jqwik, Testcontainers	Unit, property-based and integration tests
Observability	Micrometer, Prometheus, Grafana	Closes the observability gap on your resume
CI	GitHub Actions	Build, tests and the regression gate
Local stack	Docker Compose	One command to run everything
Optional stretch	Terraform for AWS	Closes the Terraform gap, only if time allows

Repository layout

```
paystruct/
  README.md                   results table, architecture, how to run
  docker-compose.yml          Kafka, Postgres, Redis, Prometheus, Grafana
  service/                    Spring Boot application
    src/main/java/.../ingest/      parsing, validation, address jobs
    src/main/java/.../rules/       AddressRule chain and ParseState
    src/main/java/.../llm/         fallback, checks, confidence
    src/main/java/.../explain/     rule-card retrieval and explanations
    src/main/java/.../pipeline/    Kafka consumers and assembler
    src/main/java/.../api/         REST controllers
    src/main/resources/countries/  one YAML profile per country
    src/main/resources/xsd/        downloaded ISO 20022 schemas
```

```
  rule-cards/                one YAML file per rule
  data/
    generator/               Python synthetic data generator
    labelling-guide.md       how ambiguous cases are labelled
    splits/                  dev, calibration, test (frozen), hard set
  eval/                      Python harness, baselines, reports
  review-ui/                 review screen and demo page
  docs/
    architecture.md
    decisions/               one short record per design decision
```

Keep a decision log. Write a short markdown record for each real choice: why at-least-once delivery, why the grounding check, why the threshold sits where it does. These records give you ready answers for system design interviews, and they show reviewers how you think.

Testing, CI/CD, deployment and cost

Testing proves the guarantees hold, not just that the code runs; the demo runs for free; and LLM spending stays under a cap you set.

Testing layers

Layer	Tool	What it proves
Unit	JUnit 5	Each `AddressRule` and each check behaves as designed
Property-based	jqwik	Across thousands of random inputs: no field ever exceeds its length, and no output value is missing from the input
Golden files	JUnit 5	Known XML in, expected XML out, for a fixed set of messages
Integration	Testcontainers	The real pipeline runs against real Kafka, Postgres and Redis containers
Security	JUnit 5	XXE payloads and oversized files are rejected
Contract	OpenAPI	The API matches its published documentation

The property-based tests are worth highlighting. They turn "no hallucinated values" from a claim into a tested invariant.

CI pipeline (GitHub Actions, on every pull request)

1. Build and run unit and property-based tests.
2. Run integration tests with Testcontainers.
3. Run the rules-only regression gate on 100 addresses.
4. Build the Docker image.

The full LLM evaluation runs manually or on a schedule, never on every push, so cost stays controlled.

Deployment

- **Live demo:** the synchronous API path and the demo page on a free or low-cost host, with a hosted Postgres. Check current free tiers when you deploy, because they change often.
- **Kafka pipeline:** benchmarked locally through Docker Compose, with the hardware stated. A hosted Kafka for a portfolio is not worth the cost. Say this openly rather than implying a cloud deployment.
- **Secrets** in environment variables only, never in the repo, as in SmartMail.
- **Monitoring:** Micrometer counters for each route, latency histograms and an LLM cost counter, shown on a Grafana dashboard. Put a screenshot in the README.

Cost control

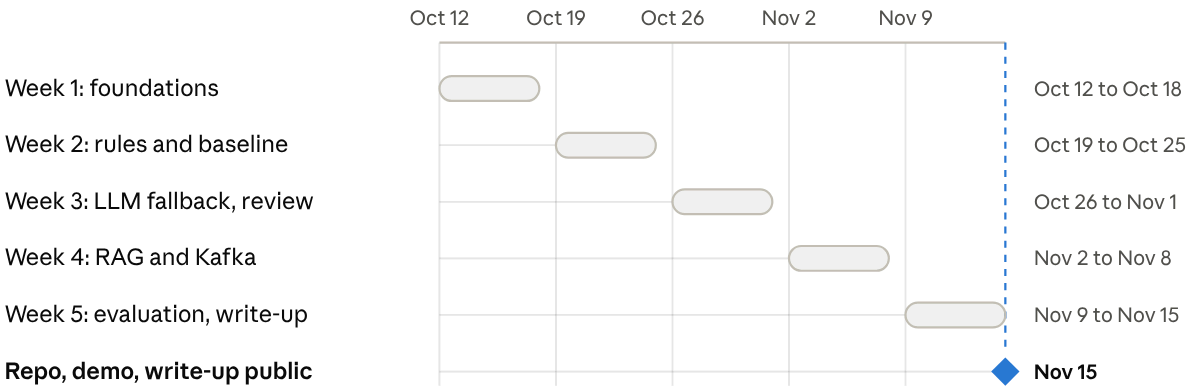
- Set a total LLM budget for the project before you start, and track spending from the cost counter.
- Use the cache and the small model by default, and run the full test set only when something meaningful has changed.
- Report the real cost per 1,000 addresses. It is a number interviewers remember.

5-week build plan

The plan runs five one-week phases from 12 October to 15 November 2026, assuming about 15 to 20 hours a week alongside interview prep. Every week ends with something working and committed, so the GitHub history shows steady recent work.

Five one-week phases end with everything public on Nov 15

PayStruct build plan by week, 2026, alongside interview prep



Build plan · 5 weeks, 12 Oct to 15 Nov 2026

If a week slips, cut scope in the order listed under Risks rather than moving the end date.

Week 1: foundations

- [ ] Create the repo, README skeleton and Docker Compose with Postgres and Redis
- [ ] Download the pacs.008 and pain.001 XSDs and generate Java classes with `xjc`
- [ ] Build safe XML parsing, type detection, XSD validation and address job extraction
- [ ] Write the labelling guide for ambiguous cases
- [ ] Build the synthetic generator for IE and GB first, with a fixed seed

Week 2: rules engine and baseline

- [ ] Implement the `AddressRule` chain and `ParseState`
- [ ] Add country profiles for all eight countries
- [ ] Generate all four dataset splits, and write the 100 hard addresses by hand
- [ ] Run libpostal and Swift's model as baselines
- [ ] Record the first rules-only results in `eval_runs`

Week 3: LLM fallback and review

- [ ] Add the LLM call with enforced JSON output
- [ ] Implement the grounding, schema, consistency and agreement checks
- [ ] Add property-based tests for the no-invented-values guarantee
- [ ] Build confidence scoring and set thresholds on the calibration split
- [ ] Build the review queue table, API endpoints and a simple review screen
- [ ] Add the Redis cache and the budget cap

Week 4: RAG and Kafka

- [ ] Write 60 to 100 rule cards
- [ ] Build keyword and vector search with rank fusion and the citation guard
- [ ] Label 100 failing messages and measure recall at 3
- [ ] Move the pipeline onto Kafka topics with retries, DLQ and the assembler
- [ ] Add Micrometer metrics and the Grafana dashboard

Week 5: evaluation, demo and write-up

- [ ] Run the full evaluation of all five systems on the frozen test set
- [ ] Plot the coverage versus accuracy curve
- [ ] Run the Kafka throughput benchmark and record the hardware
- [ ] Deploy the demo page and synchronous API
- [ ] Write the README results section from `eval_runs`
- [ ] Publish the write-up, then update your resume with the measured numbers

Risks, honesty rules and how to present it

The biggest risk is not technical: it is finishing a demo without measured results, or describing results more strongly than the evidence allows.

Risks

Risk	Effect	Mitigation
Rules tuned to the generator's habits	Inflated accuracy	Frozen test set, hand-written hard set reported separately
Swift's model is better at raw parsing	Weaker headline	Compete on the full pipeline, report the comparison honestly
Scope creep	Nothing finished before interviews	Cut in this order: Terraform, Grafana, demo animation, then extra countries
Deadline news changes again	Outdated framing	Check Swift's announcement before publishing, and date your write-up

Risk	Effect	Mitigation
Rule-card errors	Wrong explanations	Every card cites its source; spot-check the explanations
LLM costs	Budget overrun	Cache, small model, budget cap, manual full evaluations
Time against interview prep	Both suffer	Fixed weekly hours, and ship week by week, not all at the end

Honesty rules

- Say "synthetic data" every time you report a number.
- Never claim production use, bank adoption, or CBPR+ certification.
- Say "implements a documented subset of CBPR+ address rules," not "CBPR+ compliant."
- Every number on your resume must come from an `eval_runs` record you can show.
- Publish the failures section of the write-up as prominently as the wins.

Resume bullets (fill the brackets only with measured numbers)

- Built PayStruct, a Java/Spring Boot and Kafka engine that converts unstructured ISO 20022 payment addresses into CBPR+ structured format, using a rules-first pipeline with an LLM fallback; reached [X]% exact match on [N] labelled synthetic addresses across [K] countries.
- Guaranteed zero invented field values through a grounding check verified by property-based tests, and routed low-confidence results to human review, keeping errors among auto-accepted addresses at [Y]% while automating [Z]%.
- Benchmarked against libpostal and Swift's open-source structuring model; added hybrid BM25 and vector retrieval to explain validation failures with cited rules ([R]% recall at 3).

Interview talking points

1. Why rules come first, and what that saved in cost and errors, with your numbers.
2. How the grounding check makes hallucination impossible rather than unlikely.
3. How the confidence threshold was set on calibration data, and the coverage versus accuracy trade-off.
4. Why at-least-once delivery with idempotent writes instead of exactly-once.

5. Why only address lines go to the LLM: GDPR data minimisation.
6. Where Swift's model was better, and what you would do next.

The write-up. Publish one post when the project is done: the industry problem, why Swift deferred the deadline, your design, your results table, and your failures. Share it on LinkedIn, and link it from your resume next to the repo and the demo.

Sources

- Swift: ISO 20022 in bytes, the journey continues (structured address requirement, Swift's open-source structuring model)
- Swift: structured address migration extended, 27 August 2026, updated 21 September 2026
- Swift-SC iso20022-address-structuring on GitHub
- Lloyds Bank: ISO 20022 migration timelines (dated May 2025, so check it against Swift's later announcement)